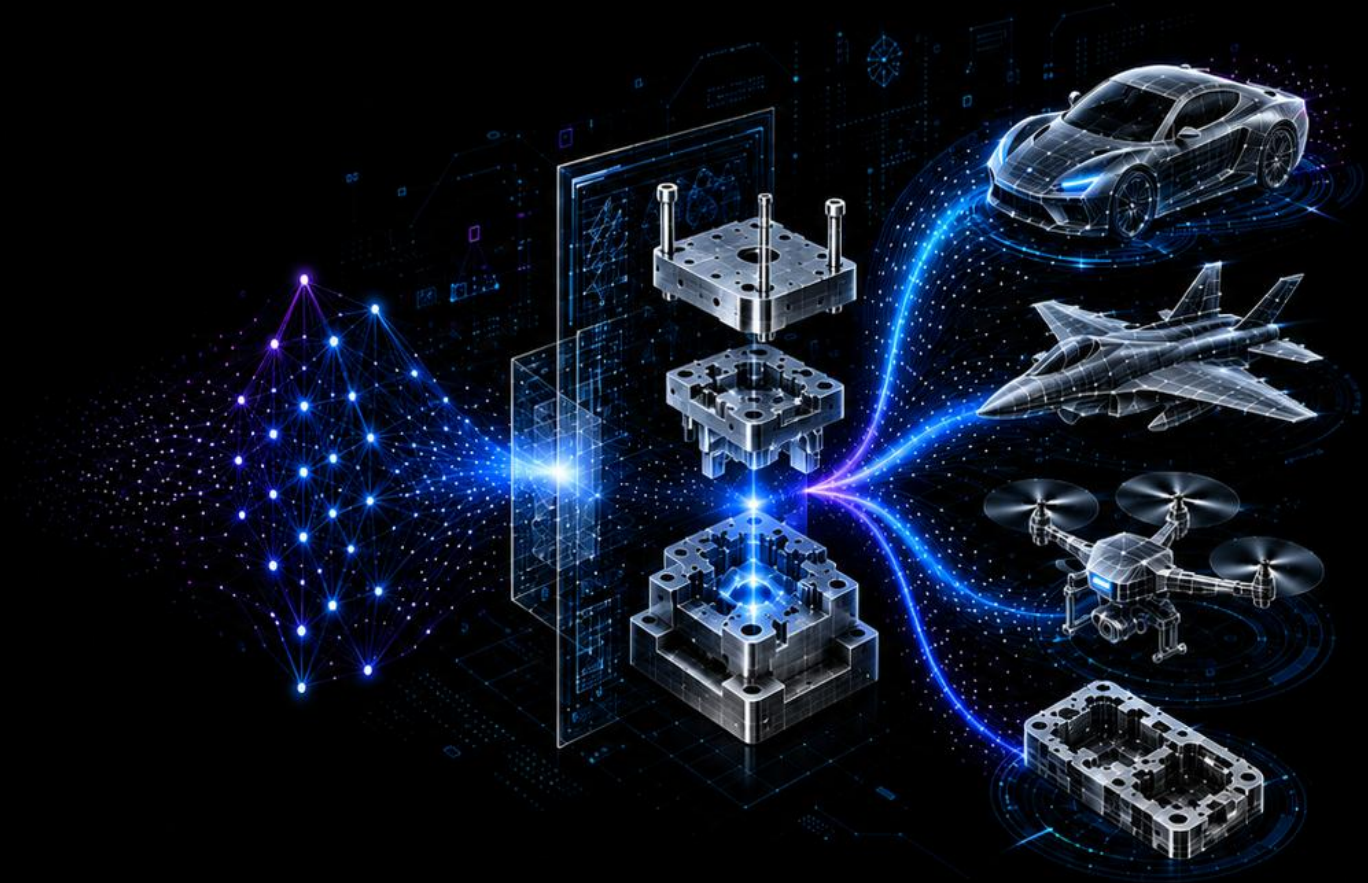


IDEA LAB AI STUDY

5장: 합성곱 신경망

최준석
(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



IDEA

Intelligent Design
for Engineering
Applications Lab

CNN 예제 1 – 추가 학습 내용

CPU → GPU

- 기존 코드는 CPU를 가정하고 코드를 작성했기 때문에 GPU로 변경 해주기 위해서 런타임 변경 후

```
print(f"Device set to: {device}")
```

해당 코드를 추가 해줌. (아래는 실제 출력 결과)

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")  
print(f"Device set to: {device}")
```

```
... Device set to: cuda:0
```

Cuda 의 의미

- cuda란?

엔비디아가 만든 프로그램으로, GPU를 사용한다는 명령어.

- cuda:0 의 의미는?

프로그래밍에서는 숫자를 0부터 세기 때문에, "0번째(즉, 가장 첫 번째) 그래픽카드"를 사용한다는 뜻.

torch.nn.Dropout

- 전체 코드 흐름에서 과적합을 방지하기 위해서

`self.drop = nn.Dropout (0.25)` 코드 사용.

의미: 25% 확률로 텐서의 값이 0이 되고, 나머지는 $(1/(1-0.25))$ 만큼 곱해진다는 뜻

계산: $\text{계수} = \frac{1}{1-0.25} = \frac{1}{0.75} \approx 1.333$

75% 확률로 살아남은 값에 1.333 만큼 곱해짐.

```
class FashionDNN(nn.Module):  
    def __init__(self):  
        super(FashionDNN, self).__init__()  
        self.fc1 = nn.Linear(in_features=784, out_features=256)  
        self.drop = nn.Dropout(0.25)  
        self.fc2 = nn.Linear(in_features=256, out_features=128)  
        self.fc3 = nn.Linear(in_features=128, out_features=10)  
  
    def forward(self, x):  
        out = x.view(-1, 784)  
        out = F.relu(self.fc1(out))  
        out = self.drop(out)  
        out = F.relu(self.fc2(out))  
        out = self.fc3(out)  
        return out
```

■ CNN 예제 1 – 추가 학습 내용

```
train = Variable(images.view(100, 1, 28, 28))
```

- 해당 코드 괄호 의미는 아래와 같음.

(배치 크기, 채널 수, 이미지 높이, 이미지 너비)

배치 크기: 100

채널 수: 1 (흑백)

이미지 높이 & 너비: 28*28 (기본 mnist 데이터 크기)

```
num_epochs = 5;
count = 0;
loss_list = [];
iteration_list = [];
accuracy_list = [];

predictions_list = [];
labels_list = [];

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1
```

count += 1의 의미

- count = count + 1 이라는 의미로,
이 코드가 반복문 안에서 실행될 때마다 숫자가 1씩 증가

- Correct 와 count 의 차이점

Correct(맞은 개수): 딥러닝 코드에서는 모델이 예측한 값과 실제 정답이 일치했을 때만 숫자를 올리는 '정답 카운터'로 주로 사용

예: `correct += 1`

Count(전체 개수): 딥러닝 코드에서는 정답과 상관없이 '지금까지 지나간 전체 데이터의 개수'나 '반복문이 몇 번 돌았는지'를 세는 용도

예: `count += 1`

```
outputs = model(train)
loss = criterion(outputs, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()

count += 1
```

■ CNN 예제 1 – 추가 학습 내용

코드 실행 중 오류

- (NameError: name 'input_data' is not defined)

교재 기준 → `out = input_data.view(-1,784)`

`def forward(self, x):`를 보면, 이 메서드는 `x` 라는 이름으로 입력을 받도록 명시되어 있다. 하지만 현재, **`input_data`는 `forward` 메서드 내부에서 정의되거나 클래스의 속성으로 정의된 적이 없음**

따라서, `input_data` 모든 스코프에서 찾을 수 없기 때문에 `NameError`가 발생하였고,
`out = input_data.view(-1,784)`을 **`out = x.view(-1,784)`로 수정**

```
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256)
        self.drop = nn.Dropout(0.25)
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, x):
        out = x.view(-1,784)
        out = F.relu(self.fc1(out))
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

교재와 정확도가 다른 이유

- 정확도가 다른 이유는 **가중치의 무작위 초기화** 때문일 가능성이 크다

가중치들의 무작위로 초기화되는 초기값에 따라 학습 경로가 달라지고 최종 정확도에도 차이가 발생한다.
교재의 결과와 사용자의 결과가 단 1~2% 정도 차이 나는 것은 이러한 무작위성 때문에 흔히 발생하는 일

교재의 정확도: 86%

실습 결과값: **88%**

```
if not (count % 500):
    print("iteration: {}, loss: {}, Accuracy: {}".format(count, loss.data, accuracy))

... iteration: 500, loss: 0.5370032787322998, Accuracy: 84.1883316040039%
iteration: 1000, loss: 0.48915961384773254, Accuracy: 86.14833068847656%
iteration: 1500, loss: 0.3595401346683502, Accuracy: 86.47166442871094%
iteration: 2000, loss: 0.4181605875492096, Accuracy: 87.04666900634766%
iteration: 2500, loss: 0.27132928371429448, Accuracy: 87.77166748846875%
iteration: 3000, loss: 0.3106651306152344, Accuracy: 88.31666564941406%
```

■ CNN 예제 2 – 추가 학습 내용

MaxPool2d 에서 출력 크기

➤ 기존 식에서의 변형 형태.

I: 입력 필터의 크기 (= Conv2d 의 출력의 크기)

F: 커널의 크기

$$\begin{aligned}\text{출력 크기} &= (I-F)/F+1 = I/F - 1 + 1 = I / F \\ &= \text{입력 필터의 크기} / \text{커널 크기}\end{aligned}$$

➤ 왜 784 / 2 인가?

MaxPool 전 과정인 Conv2d의 출력 크기가 28*28 인데 이게 곧 MaxPool의 입력의 크기이기 때문이다

← Conv2d 의 출력 크기

$$(W-F+2P)/S + 1 = (28-3+(2*1))/1 + 1 = 28$$

Fashion_minst 입력의 크기는 28 x 28

합성곱 네트워크 생성

```
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc1 = nn.Linear(in_features=64*6*6, out_features=600)
        self.drop = nn.Dropout(0.25)
        self.fc2 = nn.Linear(in_features=600, out_features=120)
        self.fc3 = nn.Linear(in_features=120, out_features=10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.drop(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out
```

■ CNN 예제 2 – 추가 학습 내용

BatchNorm2d의 의미

- BatchNorm2d는 각 채널별로 데이터를 정규화해서 학습을 더 빠르고 안정적으로 만들어주는 레이어이다.

➤ 왜 필요한가?

신경망 학습 중에 각 레이어의 입력 분포가 바뀌는 현상 발생

예를 들어 Conv2d를 통과한 출력 값들이 어떤 배치에서는 값이 크고, 어떤 배치에서는 값이 작으면 다음 레이어가 매번 다른 분포에 적응해야 해서 학습이 느려지고 불안정해진다.

BatchNorm은 이러한 현상을 막기 위해, 각 배치마다 평균 0, 분산 1로 값을 맞춰준다.

```
self.layer1 = nn.Sequential(  
    nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),  
    nn.BatchNorm2d(32),  
    nn.ReLU(),  
    nn.MaxPool2d(kernel_size=2, stride=2)  
)  
self.layer2 = nn.Sequential(  
    nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),  
    nn.BatchNorm2d(64),  
    nn.ReLU(),  
    nn.MaxPool2d(2)  
)
```

Out.size(0) 이 왜 100인가?

- 파이토치가 데이터의 크기를 물어볼 때, [배치, 채널, 높이, 너비] 순서로 대답

여기서 **size(0): 배치의 크기**

size(1): 채널 수, 필터 수

size(2): 세로 높이

size(3): 가로 너비

- `out.view(100,-1) ≠ out.view(out.size(0),-1)?`

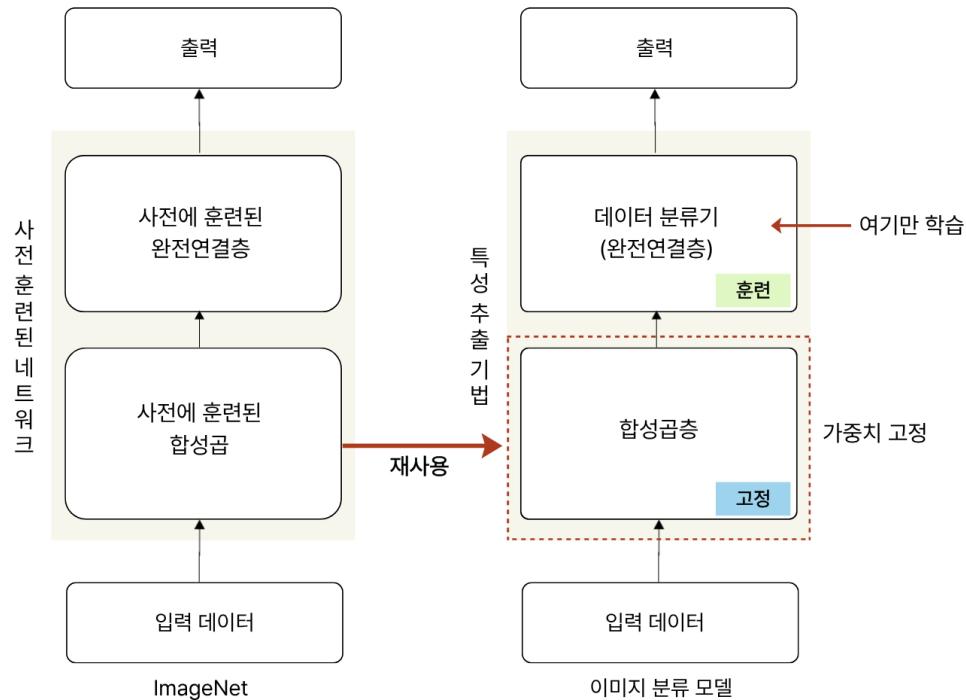
의미는 비슷하지만 코드의 유연성 때문에 `out.size` 사용

예) 학습 속도를 위해 위에서 100이 아닌 200으로 배치 크기를 바꾼다면 `out.view(100,-1)`에서는 배치 크기가 달라서 오류 발생.

■ CNN 예제 3 - 추가 학습 내용

특성 추출 기법이란?

- 사전 훈련된 모델을 가져온 후 가져온 모델의 가중치는 고정된 후, 마지막 완전 연결층(이미지 분류 단계)만 학습.



Claude로 수정한 기존 교재 코드의 수정 사항

- numpy import 누락

기존 교재에서 numpy import가 빠져 있어 np.transpose/np.array/np.arange에서 NameError 발생

<기존 코드>

```
# 라이브러리 호출
import os
import time
import copy
import glob
import cv2
import shutil

import torch
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
```

<수정 코드>

```
# 라이브러리 호출
import os
import time
import copy
import glob
import cv2
import shutil

import numpy as np # [FIX] 원본에 numpy import 누락
import torch
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
```

■ CNN 예제 3 - 추가 학습 내용

Claude로 수정한 기존 교재 코드의 수정 사항

➤ .next() 사용 (구버전 문법)

samples, labels = next(iter(train_loader)) 이와 같이
.next가 아닌 내장함수 next()를 사용

〈기존 코드〉

```
# 학습에 사용될 이미지 출력  
samples, labels = iter(train_loader).next()
```



〈수정 코드〉

```
# 학습에 사용될 이미지 출력  
samples, labels = next(iter(train_loader))  
t() 사용  
classes = {0: 'cat', 1: 'dog'}
```

Claude로 수정한 기존 교재 코드의 수정 사항

➤ 사전 학습 모델 호출

최신 파이토치에서 pretrained=True가 아닌 weights= 방식
(예: weights=models.ResNet18_Weights.DEFAULT)

〈기존 코드〉

```
# 사전 훈련된 모델 내려받기  
resnet18 = models.resnet18(pretrained=True)
```



〈수정 코드〉

```
# 사전 훈련된 모델 내려받기  
resnet18 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)  
에서 deprecated -> weights= 방식
```

■ CNN 예제 3 – 추가 학습 내용

Claude로 수정한 기존 교재 코드의 수정 사항

➤ 교재의 모델 객체 생성 및 손실함수 정의 블록 제거

실제 학습(train_model)과 이후 파라미터 수집은 전부 resnet18 객체로 진행됨. 그 결과 이 model은 한 번도 학습되지 않은 채 맨 마지막 예측 코드에서 잘못 사용됨으로 불필요 블록 제거.

```
# 모델의 파라미터 값 확인

for name, param in resnet18.named_parameters():
    if param.requires_grad:
        print(name, param.data)

# [FIX] 원본은 여기서 별도의 model 객체를 새로 만들고 optimizer/cost를 정의했지만,
# 실제 학습(train_model)과 이후 파라미터 수집은 전부 resnet18 객체로 진행됩니다.
# 그 결과 이 model은 한 번도 학습되지 않은 채 맨 마지막 예측 코드에서 잘못 사용되고 있었습니다.
# resnet18 하나만 일관되게 사용하도록 중복 블록을 제거합니다.
print(resnet18)

# 모델 객체 생성 및 손실 함수 정의
```

Claude로 수정한 기존 교재 코드의 수정 사항

➤ Pred 이진화 삭제

$\text{pred}[\text{pred} \geq 0.5] = 1$ / $\text{pred}[\text{pred} < 0.5] = 0$ 이와 같은 이진화는, 시그모이드 출력을 사용하는 이진 분류일 때 쓰는 방식이기 때문에, 해당 코드와는 상관없음.

〈기존 코드〉

```
_, preds = torch.max(outputs.data, 1)
pred[pred >= 0.5] = 1
pred[pred < 0.5] = 0
running_corrects += preds.eq(labels.cpu()).int().sum()
```



〈수정 코드〉

```
_, preds = torch.max(outputs.data, 1)

running_corrects += preds.eq(labels).int().sum()
```

■ CNN 예제 3 - 추가 학습 내용

Claude로 수정한 기존 교재 코드의 수정 사항

➤ Num_workers

: 데이터를 불러올 때 하위 프로세스를 몇 개 사용할지 결정. 너무 많은 하위 프로세스는 메모리 부족 현상을 발생시키기 때문에 colab에서 2(낮은 숫자)로 바꿈. 하지만 기존 8로 돌려도 괜찮았음.

〈기존 코드〉

```
train_loader = torch.utils.data.DataLoader(  
    train_dataset,  
    batch_size=32,  
    num_workers=2,  
    shuffle=True,  
)
```

〈수정 코드〉

```
train_loader = torch.utils.data.DataLoader(  
    train_dataset,  
    batch_size=32,  
    num_workers=8, # [COLAB] Colab 기본 런타임은 CPU 코어  
    shuffle=True,  
)
```

Claude로 수정한 기존 교재 코드의 수정 사항

➤ device 불일치

preds는 모델 출력 값으로 GPU(device) 속에 있음. 그런데 labels.cpu()는 CPU 디바이스에 속해있기 때문에, 서로 다른 디바이스에 있는 텐서끼리 비교하게 되어 에러 발생

〈기존 코드〉

```
_, preds = torch.max(outputs.data, 1)  
pred[pred >= 0.5] = 1  
pred[pred < 0.5] = 0  
running_corrects += preds.eq(labels.cpu()).int().sum()
```

〈수정 코드〉

```
running_corrects += preds.eq(labels).int().sum()  
수정 (labels는 이미 device 위에 있음)
```

Part 1

심층 신경망

Part 2

합성곱 신경망

Part 3

전이 학습

Part 4

특성 맵

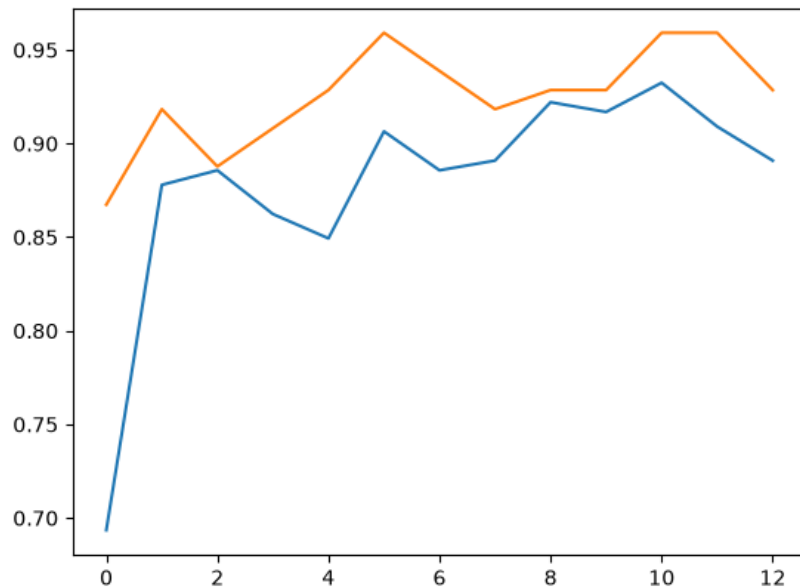
CNN 예제 3 - 추가 학습 내용

최종 결과 값

➤ 훈련과 테스트 데이터에 대한 정확도

(주황: 테스트 데이터, 파랑: 훈련 데이터)

[y축: 정확도(0~1 사이) / x축: epoch (학습 반복 횟수 0~12)]

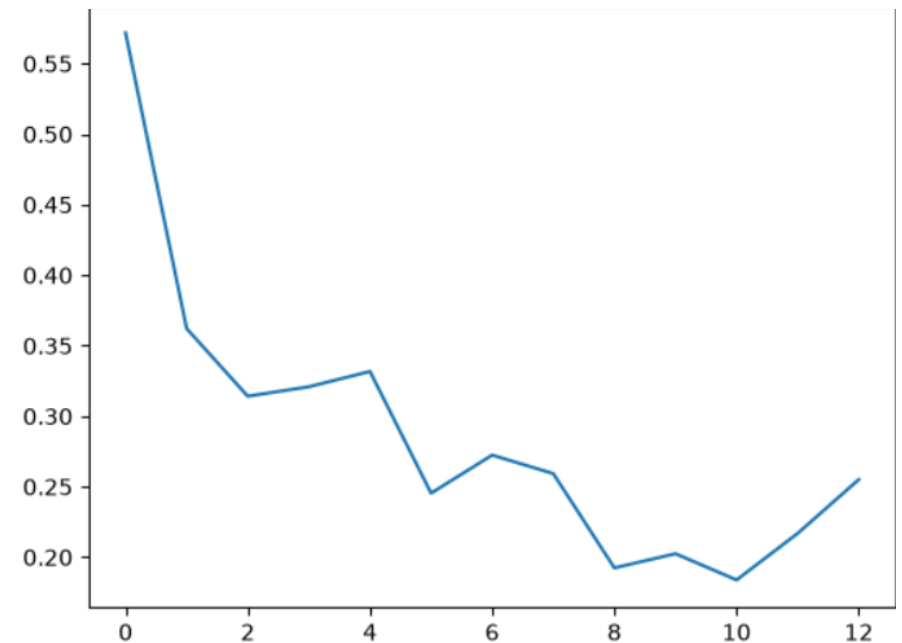


=> 훈련 정확도보다 테스트 정확도가 전반적으로 더 높게 나오는 것을 보았을 때, 테스트셋이 상대적으로 쉽거나 데이터 수가 적어 변동성이 크다고 볼 수 있다

최종 결과 값

➤ 훈련 데이터에 대한 오차

[y축: 훈련 손실(loss) 값 / x축: epoch, 0~12]



=> 전반적으로 우하향 하지만 배치 크기도 작고 에폭 범위도 적어 진동이 존재한다.

CNN 예제 3 - 추가 학습 내용

최종 결과 값

➤ 개와 고양이에 대한 예측 결과

수정 후 코드는 에러 없이 끝까지 실행 했음.

385장의 소규모 데이터셋만으로 사전 학습된 ResNet18을 통해 테스트 정확도 95.92%를 얻음



■ CNN 예제 4 - 추가 학습 내용

특성 맵이란?

➤ 원본 이미지를 여러 합성곱 층에 통과시키면서 "이 사진에서 어떤 패턴이 있는지"를 숫자로 뽑아냄. 이러한 중간 숫자 배열들을 이미지로 형상화 한 것.

➤ 왜 사용하는가?

각 층에서 "이미지의 어떤 부분에 반응하는지"를 확인하기 위해서

❖ 초반 층의 특성맵 → 보통 테두리(edge), 선, 색깔 경계 같은 단순한 패턴에 반응하는 게 보임

❖ 후반 층의 특성맵 → 귀 모양, 눈, 털 무늬처럼 좀 더 복잡하고 추상적인 패턴에 반응하는 게 보임

즉, "해당 모델이 사진을 보고 무엇에 집중하는지"를 사람이 직관적으로 확인하는 디버깅/해석 방법

보간법

➤ 보간법

: 이미지 크기를 바꿀 때, 원래 없던 픽셀 값을 주변 픽셀들을 이용해 새로 "추정해서 채워 넣는" 방법.

➤ 여러 방식

1. Nearest neighbor (최근접 이웃)

새 픽셀 자리에서 제일 가까운 원본 픽셀 값을 그대로 복사하는 방법은 빠르지만 결과가 계단처럼 각지고 거칠게 보임. (블록 현상).

2. Bilinear (이중 선형 보간) — 가장 흔히 쓰는 기본값

새 픽셀 주변의 원본 픽셀 4개(2×2)를 놓고, 거리에 비례해서 가중 평균을 냄.

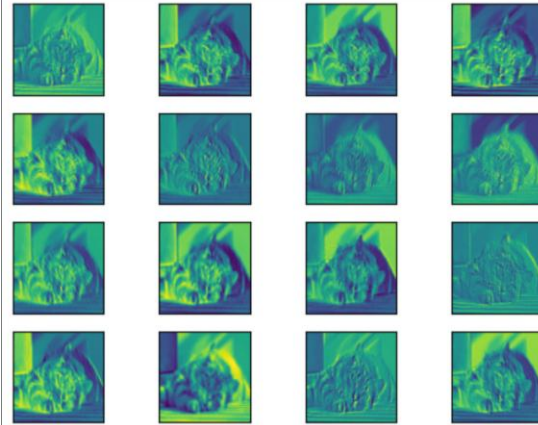
3. Bicubic (삼차 보간)

주변 픽셀 4×4 (16개)를 이용해 곡선 형태로 더 정교하게 추정 bilinear보다 더 부드럽고 선명하지만 계산량이 더 많음.

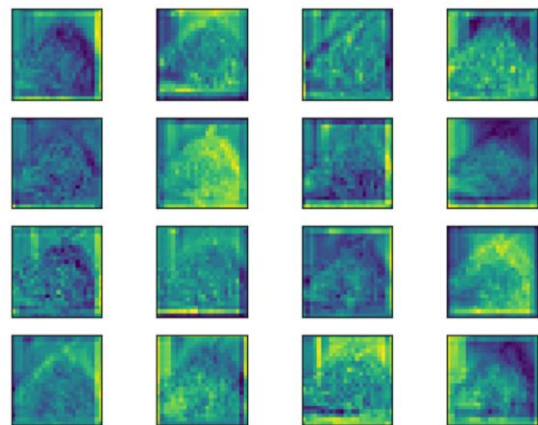
CNN 예제 4 - 추가 학습 내용

특성 맵 결과 사진

➤ 첫 번째 계층

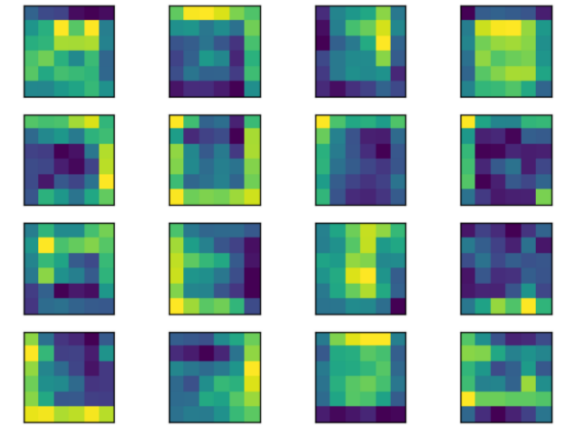


➤ 20번째 계층



특성맵 결과 사진

➤ 40번째 계층



- 출력층에 가까워질 수록 원래의 형태보다는, 이미지 특징들 중심으로 전달.
- 이와 같이 CNN의 내부 구조를 시각화 함으로써, CNN 결과의 신뢰성을 확보.

Any Questions?

이름 최준석

소속 한양대학교 기계공학과

이메일 chw10109@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

